D단계 (원두→커피, 인증·세션 관리) 풀이집

실행 방법 (문제를 풀기 전에 먼저 하십시오)

이 문제들은 실제로 동작하는 서버이기 때문에, 풀기 전에 먼저 서버를 실행시켜야 합니다. 서버를 한 번만 실행하면 4장 10문제가 전부 그 서버 안에서 함께 동작하므로, 문제를 옮길 때마다 서버를 새로 켤 필요는 없습니다.

1. Node.js 설치 확인

PowerShell을 열고 아래 명령을 실행합니다.

```
node --version
```

버전 번호가 나오면 설치되어 있는 겁니다. "인식할 수 없다"는 오류가 나오면 nodejs.org에서 LTS 버전을 설치한 뒤 PowerShell을 새로 열어서 다시 시도합니다.

2. 통합 서버 실행

VSCode에서 `wargame problems` 폴더(이 `problems` 폴더의 바로 위 폴더)를 작업 폴더로 열고 터미널을 켜면, 아래 명령들이 그대로 통합니다(상대 경로라서 `wargame problems` 폴더를 어디로 옮기든 그대로 작동합니다). PowerShell 창을 하나만 열어서, 아래처럼 `d-auth-session` 폴더로 이동한 뒤 통합 서버를 실행합니다.

```
cd "problems\d-auth-session"
node combined-server.js
```

실행 중이라는 메시지가 뜨면(`D단계 통합 서버 실행: http://localhost:4000/`), 그 창은 그대로 켜 둔 채로 브라우저에서 각 문제 주소(`http://localhost:4000/ch4/번호/`)에 접속하면 됩니다. 서버를 끄려면 그 창에서 `Ctrl+C`를 누릅니다.

배포된 사이트로 바로 확인하기

로컬 서버를 직접 띄우지 않고, 이미 배포된 사이트(`https://kload-wargame.onrender.com`)에서 바로 따라 해도 됩니다. 아래 모든 안내에서 `http://localhost:4000`을 `https://kload-wargame.onrender.com`으로 바꿔서 접속하면 똑같이 동작합니다. ("서버가 실행되면..." 같은 로컬 서버 실행 안내 문장은 이 경우 무시하면 됩니다.)

3. 전체 문제·경로 목록

문제	폴더	경로
D1	`01-bean-core`	`/ch4/01/`
D2	`02-batch-idor`	`/ch4/02/`
D3	`03-approval-skip`	`/ch4/03/`
D4	`04-nest-csrf`	`/ch4/04/`
D5	`05-otp-bruteforce`	`/ch4/05/`
D6	`06-logout-ghost`	`/ch4/06/login`
D7	`07-hidden-debug`	`/ch4/07/`
D8	`08-jwt-alg-confusion`	`/ch4/08/`
D9	`09-oauth-state`	`/ch4/09/`
D10	`10-2fa-bypass`	`/ch4/10/login`

4장 · 01 (하) — 원두 몬스터 추출 시스템

풀이 흐름: 로그인합니다 → "핵심" 패널이 잠겨 있는 것을 확인합니다 → 브라우저 쿠키 `d01_bean_state`를 `raw`에서 `roasted`로 수정합니다 → 재접속해 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\d-auth-session"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:4000/ch4/01/` 입니다.

1. 주어진 계정(`roaster` / `coffee123`)으로 로그인합니다.
2. 개발자 도구(F12 또는 Ctrl+Shift+I) → Application → Cookies에서 `d01_bean_state` 값을 `roasted`로 바꿉니다.
3. 새로고침하면 "핵심(로스팅 완료)" 패널이 열리며 플래그가 나옵니다.

플래그: `BREW{ch4_01_b3an_r0ast_bypass}`

취약점: 쿠키에 담긴 상태값을 서버가 검증 없이 그대로 믿습니다 (클라이언트 측 값 신뢰).

4장 · 02 (하) — 배치 추적 시스템

풀이 흐름: 로그인합니다 → 내 담당 배치(#7)는 평범한 기록임을 확인합니다 → URL의 배치 번호를 1로 바꿔봅니다 → 핵심 배치(몬스터 본체) 기록에서 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\d-auth-session"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:4000/ch4/02/` 입니다.

1. 주어진 계정(`roaster` / `coffee123`)으로 로그인합니다.
2. 대시보드에서 "내 담당 배치"가 `/ch4/02/batch?id=7`로 연결되어 있는 것을 확인합니다.
3. 주소창에서 `id=7`을 `id=1`로 바꿔 접속합니다.
4. 배치 #1 "핵심 배치(몬스터 본체)" 기록에서 플래그가 나옵니다.

플래그: `BREW{ch4_02_idor_batch_snoop}`

취약점: 서버가 로그인 여부만 확인하고, 요청한 배치 번호가 로그인한 사람의 것인지는 확인하지 않습니다 (IDOR — Insecure Direct Object Reference).

4장 · 03 (하) — 핵심 배치 폐기 승인 시스템

풀이 흐름: 로그인합니다 → 폐기 절차가 1차 검토·2차 승인 2단계로 나뉘어 있는 것을 확인합니다 → 1차 검토를 제출하지 않은 채 2차 승인 주소로 직접 이동합니다 → 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\d-auth-session"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:4000/ch4/03/` 입니다.

1. 주어진 계정(`roaster` / `coffee123`)으로 로그인합니다.
2. "폐기 절차 안내" 화면에서 1차 검토·2차 승인 2단계 절차를 확인합니다.
3. 1차 검토 페이지의 "검토 제출" 버튼을 누르지 않고, 주소창에 `/ch4/03/approve`를 직접 입력해 이동합니다.
4. "2차 승인 — 배치 #1 폐기 완료" 화면에서 플래그가 나옵니다.

플래그: `BREW{ch4_03_skip_approval_step}`

취약점: 서버가 2차 승인 요청을 처리할 때 로그인 여부만 확인하고, 1차 검토가 먼저 완료되었는지는 전혀 확인하지 않습니다 (절차 생략 — Missing step enforcement / Broken workflow).

4장 · 04 (중) — 둥지 봉인 시스템

풀이 흐름: 로그인합니다 → "둥지 봉인은 `GET /ch4/04/seal-nest`로만 처리된다"는 안내문을 확인합니다 → 열리자마자 그 주소로 자동 이동하는 HTML 페이지를 만들어 로그인된 상태에서 엽니다 → 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\d-auth-session"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:4000/ch4/04/` 입니다.

1. 주어진 계정(`roaster` / `coffee123`)으로 로그인합니다.
2. "둥지 봉인 대기 중" 화면에서 `GET /ch4/04/seal-nest`라는 내부 API 경로를 확인합니다.
3. 아래 내용으로 HTML 파일을 만들어 저장합니다.

```
<!DOCTYPE html>
<html>
<head>
<title>free coffee coupon</title>
<meta http-equiv="refresh" content="0; url=http://localhost:4000/ch4/04/seal-nest">
</head>
<body><p>loading...</p></body>
</html>
```

1. 로그인 상태를 유지한 채 이 파일을 더블클릭해서 엽니다.
2. 열리자마자 `/ch4/04/seal-nest`로 자동 이동되며 "둥지 봉인 완료" 화면과 함께 플래그가 나옵니다.

플래그: `BREW{ch4_04_csrf_seal_nest}`

취약점: 서버가 둥지 봉인 요청을 처리할 때 로그인 세션만 확인하고, CSRF 토큰은 확인하지 않습니다. 다만 "정말 외부에서 온 요청인지"는 브라우저가 자동으로 보내는 표준 헤더 `Sec-Fetch-Site`로 판정합니다 — 주소창에 직접 입력하거나(`none`) 사이트 안에서 이동하면(`same-origin`) 거부되고, 외부 HTML 파일에서 자동으로 이동한 경우(`cross-site`)만 통과합니다. 그래서 반드시 위 4-5번처럼 별도 HTML 파일을 만들어 열어야 하며, 이 주소를 브라우저 주소창에 직접 입력하는 것만으로는 통하지 않습니다. 상태를 바꾸는 작업을 GET 요청으로 처리하는 것 자체도 결함입니다 (CSRF — Cross-Site Request Forgery).

4장 · 05 (중) — 창고 긴급 개방 시스템

풀이 흐름: 공개된 계정(`headroaster` / `roast2026`)으로 로그인합니다 → "창고 긴급 개방"에 4자리 보안 코드가 추가로 필요한 것을 확인합니다 → 몇 번을 틀려도 막히지 않는 것을 확인하고, 0000-9999를 자동으로 전부 시도하는 스크립트를 돌립니다 → 맞는 코드로 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\d-auth-session"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:4000/ch4/05/` 입니다.

1. `headroaster` / `roast2026`으로 로그인합니다. "보안 코드 확인" 화면으로 이동합니다.
2. 아무 코드나 입력해 "확인"을 눌러 보고, 틀려도 잠기거나 막히지 않는 것을 확인합니다.
3. PowerShell에서 아래 스크립트를 실행해 0000부터 9999까지 코드를 자동으로 전부 시도합니다.

```
$session = $null
Invoke-WebRequest -Uri "http://localhost:4000/ch4/05/login" -Method POST -Body
@{id="headroaster"; pw="roast2026"} -SessionVariable session -UseBasicParsing |
Out-Null
for ($i = 0; $i -le 9999; $i++) {
    $code = $i.ToString("0000")
    $resp = Invoke-WebRequest -Uri "http://localhost:4000/ch4/05/verify-code"
-Method POST -Body @{code=$code} -WebSession $session -UseBasicParsing
    if ($resp.Content -match "BREW\{[^}]*\}") {
```

```
            Write-Host "맞는 코드: $code"
            Write-Host $matches[0]
            break
        }
    }
```

1. 맞는 코드가 출력되며 그 응답 안에 플래그가 함께 나옵니다.

플래그: `BREW{ch4_05_otp_bruteforce}`

취약점: 2단계 인증 코드가 4자리(0000-9999, 1만 가지)뿐이고, 틀린 시도 횟수를 서버가 전혀 세거나 제한하지 않습니다. 코드 공간이 좁고 잠금 장치도 없으니, 자동화 스크립트로 전부 시도하면 반드시 뚫립니다 (2단계 인증 자체가 있다고 안전한 것이 아니라, 그 값 공간과 시도 횟수 제한까지 갖춰야 의미가 있다는 사례).

4장 · 06 (중) — 배전반 로그아웃

풀이 흐름: 로그인해서 세션 쿠키 값을 미리 복사해둡니다 → 로그아웃합니다 → 복사해둔 쿠키 값을 다시 넣어봅니다 → 여전히 로그인된 상태로 접속되는 것을 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\d-auth-session"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:4000/ch4/06/login` 입니다.

1. 주어진 계정(`roaster` / `coffee123`)으로 로그인합니다.
2. 개발자 도구(F12 또는 Ctrl+Shift+I) → Application → Cookies에서 `d06_sid` 값을 복사해둡니다.
3. 화면의 "로그아웃" 링크를 클릭합니다.
4. 쿠키 `d06_sid`를 3번에서 복사해둔 값으로 다시 설정하고 `/ch4/06/mypage`에 접속합니다.
5. 로그아웃했는데도 여전히 "내 작업 기록" 화면이 보이며 플래그가 나옵니다.

플래그: `BREW{ch4_06_logout_ghost_session}`

취약점: 로그아웃 처리가 브라우저 쪽 쿠키만 지우고, 서버가 그 세션을 실제로 무효화하지 않습니다. 로그아웃 전에 세션 값을 알고 있던 사람은 로그아웃 후에도 계속 그 세션을 재사용할 수 있습니다.

4장 · 07 (중) — 유지보수 포털

풀이 흐름: `robots.txt`를 열어 "들어오지 말라"고 적힌 숨은 경로를 확인합니다 → 그 경로에 들어가 테스트용 관리자 계정을 발견합니다 → 그 계정으로 로그인해 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\d-auth-session"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:4000/ch4/07/` 입니다.

1. 주소창에 `http://localhost:4000/ch4/07/robots.txt`를 입력해 엽니다. `Disallow: /ch4/07/debug-staff-only`라는 줄이 보입니다.
2. 이 줄이 가리키는 `http://localhost:4000/ch4/07/debug-staff-only`로 직접 접속합니다.
3. 페이지에 남아있는 테스트 계정(아이디/비밀번호)을 확인합니다.
4. 그 계정으로 `/ch4/07/login`에서 로그인하면 플래그가 나옵니다.

플래그: `BREW{ch4_07_hidden_admin_creds}`

취약점: 검색엔진에게 "들어오지 말라"고 알려주는 `robots.txt`가, 역설적으로 "여기 뭔가 숨겨져 있다"는 지도 역할을 합니다. 그 경로에 테스트용 관리자 계정을 지우지 않고 그대로 남겨뒀습니다 (하드코딩된 계정 노출).

4장 · 08 (상) — 서명 토큰 발급 시스템

풀이 흐름: 로그인해서 받은 토큰이 진짜 서명(RS256)으로 되어있는 것을 확인합니다 → 서버가 공개하고 있는 공개키를 가져옵니다 → 토큰의 서명 방식을 HS256으로 바꾸고, 그 공개키 문자열을 "비밀키"처럼 사용해 가짜 서명을 만듭니다 → 서버가 속아서 admin 권한으로 인식하는 것을 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\d-auth-session"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:4000/ch4/08/` 입니다.

1. 주어진 계정(`roaster` / `coffee123`)으로 로그인합니다. `d08_auth` 쿠키에 토큰이 발급됩니다.
2. `http://localhost:4000/ch4/08/public_key.pem`에 접속해 "공개키" 상자 안의 `-----BEGIN PUBLIC KEY-----`로 시작해 `-----END PUBLIC KEY-----`로 끝나는 텍스트만 마우스로 드래그해서 복사합니다(화면 전체를 복사하면 제목 글자까지 섞여 들어가니 주의하십시오). **주의:** 서버는 이 PEM 문자열 끝의 줄바꿈(개행문자) 하나까지 포함한 값을 서명 키로 씁니다. 마우스 드래그로는 끝 개행이 복사 안 될 수 있으니, 5번에서 서명이 안 맞으면 복사한 문자열 맨 끝에 개행문자(`\n`) 하나를 수동으로 추가해서 다시 시도해 보십시오.
3. 토큰의 가운데 부분(payload)을 디코드해 `role`을 `admin`으로 바꾸고, 헤더 부분(header)의 `alg`를 `"RS256"`에서 `"HS256"`으로 바꿉니다.
4. 바뀐 헤더.바뀐 payload 문자열을, 3번에서 복사한 공개키 전체 문자열을 "비밀키"로 사용해 HMAC-SHA256으로 서명합니다.
5. 새로 만든 서명을 이어붙여 `d08_auth` 쿠키를 교체하고 새로고침합니다.
6. "전체 통제 — admin 권한" 화면에서 플래그가 나옵니다.

플래그: `BREW{ch4_08_jwt_alg_confusion}`

취약점: 서버가 공개키(RS256)로 서명하는 진짜 토큰 체계를 쓰지만, 검증 코드가 "토큰 안에 적힌 알고리즘 이름"을 그대로 믿고 검증 방식을 고릅니다. 알고리즘을 HS256(비밀키 하나로 서명하는 방식)으로 바꿔치기하고, 누구나 볼 수 있는 공개키 문자열을 그 "비밀키" 자리에 넣어 서명하면, 서버가 같은 공개키로 검증하다가 "맞다"고 착각합니다(RS256/HS256 알고리즘 혼동 공격). 서명 자체가 아예 없는 토큰 위조 사례보다 한 단계 더 깊은 구조 이해가 필요합니다.

4장 · 09 (상) — 제휴 계정 연동 시스템

풀이 흐름: 내 계정으로 제휴 연동을 시작해 콜백 주소를 받습니다 → 이 주소를 "문의(링크 신고)" 기능으로 관리자에게 전달합니다 → 관리자가 그 링크를 열람하면서 관리자 계정에 내 외부 계정이 연동됩니다 → 그 외부 계정 코드로 다시 로그인해 관리자 권한을 얻습니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\d-auth-session"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:4000/ch4/09/` 입니다.

1. 주어진 계정(`roaster` / `coffee123`)으로 로그인합니다.
2. "제휴 계정 연동" 링크를 눌러 콜백 주소로 이동합니다. **주소창(URL 표시줄)**에 있는 전체 주소(`http://localhost:4000/ch4/09/oauth/callback?code=...&state=...`)를 복사해둡니다. 화면 본문에 표시되는 주소는 상대 경로(`/ch4/09/oauth/callback?...`)라서, 그걸 그대로 복사해 신고하면 "올바른 콜백 주소가 아닙니다"로 거부됩니다.
3. "링크 신고" 메뉴로 이동해, 2번에서 복사한 전체 주소(절대 URL)를 붙여넣고 신고를 제출합니다.
4. 몇 초 기다립니다(관리자가 신고된 링크를 확인하는 시간입니다).
5. "외부 계정 코드로 로그인" 메뉴로 이동해, 2번 주소의 `code` 값을 입력하고 로그인합니다.
6. 관리자 전용 화면으로 들어가져 플래그가 나옵니다.

플래그: `BREW{ch4_09_oauth_state_missing}`

취약점: 제휴 계정 연동 콜백이 위조 방지용 `state` 값을 발급은 하지만, 실제로 콜백을 처리할 때 그 값이 맞는지 전혀 검증하지 않습니다. 그래서 공격자가 자기 자신의 인가 코드로 만든 콜백 주소를, 관리자가 열람하게만 만들면 관리자 세션에 공격자의 외부 계정이 그대로 연동되어 버립니다. 그 뒤로는 공격자가 자기 외부 계정으로 로그인할 때마다 관리자 계정에 들어갈 수 있습니다 (OAuth 계정 연동 탈취).

4장 · 10 (상) — 최종 제어실 (4장·전체 게임 마지막 문제)

풀이 흐름: 계정 1단계(비밀번호) 로그인만 합니다 → 2단계(보안 코드)는 서버가 발급한 실제 코드를 정상 경로로는 알 수 없어 정석으로 통과할 수 없음을 확인합니다 → 그래서 2단계를 건너뛰고, 입력하지 않은 채 최종 제어실 주소로 바로 이동합니다 → 2단계를 건너뛰었는데도 들어가지는 것을 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\d-auth-session"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:4000/ch4/10/login` 입니다.

1. `headroaster` / `roast2026`으로 1단계 로그인을 합니다. "2단계(보안 코드)" 화면으로 넘어갑니다.
2. 보안 코드를 입력하지 않고, 주소창에 직접 `http://localhost:4000/ch4/10/final-control`을 입력해 이동합니다.

3. "최종 제어실 — 공장 전체 장악 완료" 화면에서 전체 게임의 최종 플래그가 나옵니다.

플래그: `BREW{ch4_10_2fa_logic_bypass}`

취약점: 서버가 "1단계(비밀번호) 통과"와 "2단계(보안 코드) 통과"를 서로 다른 값으로 구분해서 관리해야 하는데, 최종 제어실 접근을 검사하는 코드가 실수로 1단계 통과 값만 확인합니다. 2단계 보안 코드 제출 처리 자체는 올바르게 동작합니다. 로그인(1단계 통과) 시점에 서버가 메모리에 발급한 코드와 제출값이 정확히 일치할 때만 2단계를 통과시키며, 그 코드는 정상적인 경로로는 사용자에게 보여 주거나 알려 주지 않기 때문에 정석대로 2단계를 뚫는 것은 사실상 불가능합니다. 문제는 최종 제어실 접근을 검사하는 코드가 실수로 1단계 통과 값만 확인한다는 점입니다. 그래서 2단계 인증 화면 자체는 존재하지만, 2단계를 아예 건너뛰고 주소만 알면 그냥 들어가집니다. 2단계 인증 기능이 있다는 것과, 그 통과 여부를 확인하는 코드가 올바르게 짜여 있다는 것은 별개라는 걸 보여주는 사례입니다.